Practical - 3

//WRITE A CODE FOR PROGRAM WITH URL, URL CONNECTION AND DATA GRAMS

CODE :

URL + URLConnection Program :

```
import java.net.*;
import java.io.*;

public class URLDemo {
public static void main(String[] args) throws Exception {

    URL url = new URL("https://example.com");
    URLConnection con = url.openConnection();

    BufferedReader br = new BufferedReader(
        new InputStreamReader(con.getInputStream())
    );

    String line;
    while ((line = br.readLine()) != null) {
      System.out.println(line);
    }
    br.close();
  }
}
```

UDP Server Program :

```
import java.net.*;

public class UDPServer {
public static void main(String[] args) throws Exception {

    DatagramSocket ds = new DatagramSocket(4000);

    byte[] b = new byte[1024];
    DatagramPacket dp = new DatagramPacket(b, b.length);

    ds.receive(dp);

    String msg = new String(dp.getData(), 0, dp.getLength());
    System.out.println("Received: " + msg);

    ds.close();
  }
}
```

OUTPUT :

```
Administrator: C:\WINDOWS\system32\cmd.exe - java url
:\ACAD_UU\2023_even\Course File\JAVA_Lab\Fileprogram\p3>javac url.java

:\ACAD_UU\2023_even\Course File\JAVA_Lab\Fileprogram\p3>java url
!doctype html>
html>
head>
    <title>Example Domain</title>

    <meta charset="utf-8" />
    <meta http-equiv="Content-type" content="text/html; charset=utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <style type="text/css">
    body {
        background-color: #f0f0f2;
        margin: 0;
        padding: 0;
        font-family: -apple-system, system-ui, BlinkMacSystemFont, "Segoe UI", "Open Sans", "Helvetica Neu
 Helvetica, Arial, sans-serif;

    }
    div {
        width: 600px;
        margin: 5em auto;
        padding: 2em;
        background-color: #fdfdff;
        border-radius: 0.5em;
        box-shadow: 2px 3px 7px 2px rgba(0,0,0,0.02);
    }
    a:link, a:visited {
        color: #38488f;
        text-decoration: none;
    }
    @media (max-width: 700px) {
        div {
            margin: 0 auto;
            width: auto;
        }
    }
    </style>
/head>

body>
div>
```

Practical - 4

// WRITE A JDBC CODE FOR SELECT, INSERT, DELETE OPERATIONS ON A DATABASE.

CODE :

```
import java.sql.*;

public class JDBCDemo {
   public static void main(String[] args) throws Exception {

      Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/test", "root", ""
      );

      Statement st = con.createStatement();
      st.executeUpdate("INSERT INTO student VALUES(1, 'Ram')");

      ResultSet rs = st.executeQuery("SELECT * FROM student");

      while (rs.next()) {
         System.out.println(rs.getInt(1) + " " + rs.getString(2));
      }

      st.executeUpdate("DELETE FROM student WHERE id=1");
      con.close();
   }
}
```

OUTPUT:

```
ID    NAME
1     Ram
2     Shyam
3     Mohan
```

```
1 row deleted successfully
```

```
ID    NAME
2     Shyam
3     Mohan
```

Practical - 5

// WRITE A JDBC CODE FOR PREPARED STATEMENTS, SCROLLABLE AND UPDATABLE RESULT .

CODE :

```
import java.sql.*;
public class PreparedDemo {
    public static void main(String[] args) throws Exception {
        Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/test", "root", ""
        );
        PreparedStatement ps = con.prepareStatement(
                "INSERT INTO student VALUES(?, ?)"
        );
        ps.setInt(1, 2);
        ps.setString(2, "Shyam");
        ps.executeUpdate();
        Statement st = con.createStatement(
                ResultSet.TYPE_SCROLL_INSENSITIVE,
                ResultSet.CONCUR_UPDATABLE
        );
        ResultSet rs = st.executeQuery("SELECT * FROM student");
        if (rs.first()) {
            rs.updateString(2, "UpdatedName");
            rs.updateRow();
        }
        if (rs.last()) {
            System.out.println("Last: " + rs.getString(2));
        }
        rs.close();
        st.close();
        ps.close();
        con.close();
    }
}
```

OUTPUT:

```
Last Record: Shyam
First Record: Ram
```

```
ID    NAME
1     UpdatedName
2     Shyam
```

Practical - 6

// WRITEA SERVLET PROGRAM TO READ THE PARAMETERS .

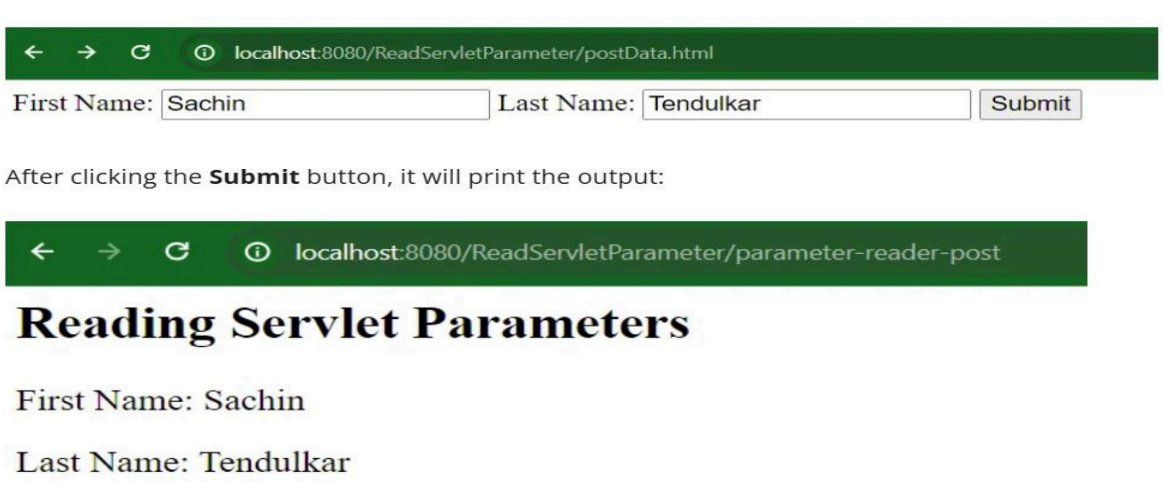
Servlet Code :

```
import java.io.IOException;
import java.io.PrintWriter;

import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/parameter-reader")
public class ParameterReaderServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {

        // Set response type
        response.setContentType("text/html");

        // Get writer
        PrintWriter out = response.getWriter();

        // Read parameters
        String firstName = request.getParameter("firstName");
        String lastName = request.getParameter("lastName");

        // Output HTML
        out.println("<html><body>");
        out.println("<h2>Reading Servlet Parameters</h2>");

        out.println("<p>First Name: " +
                (firstName != null ? firstName : "Not provided") +
                "</p>");

        out.println("<p>Last Name: " +
                (lastName != null ? lastName : "Not provided") +
                "</p>");

        out.println("</body></html>");
    }
}
```

HTML Form (Client Side) : :

```
<!DOCTYPE html>
<html>
<head>
   <title>Servlet Form</title>
</head>
<body>

   <form action="parameter-reader" method="GET">
      First Name: <input type="text" name="firstName" /><br><br>
      Last Name: <input type="text" name="lastName" /><br><br>
      <input type="submit" value="Submit" />
   </form>

</body>
</html>
```

OUTPUT:



Practical - 7

// WRITEA SERVLET CODE FOR USE OF COOKIES.

Servlet Code :

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class CookieServlet extends HttpServlet {
   public void doGet(HttpServletRequest req, HttpServletResponse res)
         throws IOException {

      res.setContentType("text/html");

      PrintWriter out = res.getWriter();
      Cookie c = new Cookie("user", "Ram");
      res.addCookie(c);

      out.println("<html><body>");
      out.println("<h2>Servlet Cookies Demo</h2>");
      out.println("<p>Cookie Added Successfully</p>");
      out.println("</body></html>");
   }
}
```

HTML Form Code :

```
<!DOCTYPE html>
<html>
<head>
   <title>Cookie Servlet Example</title>
</head>
<body>

   <h2>Servlet Cookies Demo</h2>

   <form action="cookie" method="GET">
      <label>Click to create and view cookies:</label><br><br>
      <input type="submit" value="Generate Cookie">
   </form>

</body>
</html>
```

web.xml

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
        version="3.1">

   <servlet>
      <servlet-name>CookieServlet</servlet-name>
      <servlet-class>CookieServlet</servlet-class>
   </servlet>

   <servlet-mapping>
      <servlet-name>CookieServlet</servlet-name>
      <url-pattern>/cookie</url-pattern>
   </servlet-mapping>

</web-app>
```

OUTPUT :

http://localhost:8080/YourProjectName/cookie

```
Cookie Created Successfully

Available Cookies:
Name: username
Value: Ram
```

Practical - 8

// WRITE A CODE FOR SESSION TRACKING IN SERVLET.

Servlet Code (SessionServlet):

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class SessionServlet extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        HttpSession session = request.getSession();

        session.setAttribute("username", "Ram");

        String user = (String) session.getAttribute("username");

        out.println("<html><body>");
        out.println("<h2>Session Created Successfully</h2>");
        out.println("<h3>Session ID: " + session.getId() + "</h3>");
        out.println("<h3>User Name Stored in Session: Ram</h3>");
        out.println("<h3>Retrieved from Session: " + user + "</h3>");
        out.println("</body></html>");
    }
}
```

HTML Form Code :

```
<!DOCTYPE html>
<html>
<head>
    <title>Session Tracking</title>
</head>
<body>
    <h2>Servlet Session Tracking Demo</h2>
    <form action="session" method="GET">
        <input type="submit" value="Create Session">
    </form>

</body>
</html>
```

web.xml

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
        version="3.1">

    <servlet>
        <servlet-name>SessionServlet</servlet-name>
        <servlet-class>SessionServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>SessionServlet</servlet-name>
        <url-pattern>/session</url-pattern>
    </servlet-mapping>

</web-app>
```

OUTPUT :

```
Session Created Successfully
Session ID: 5F2A8C9D12B34E
User Name Stored in Session: Ram
Retrieved from Session: Ram
```

Practical - 9

// WRITE A PROGRAM FOR CALLING ONE SERVLET BY ANOTHER.

FirstServlet (Forward to SecondServlet)

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class FirstServlet extends HttpServlet {

   public void doGet(HttpServletRequest req, HttpServletResponse res)
         throws ServletException, IOException {

      RequestDispatcher rd = req.getRequestDispatcher("second");
      rd.forward(req, res);
   }
}
```

SecondServlet (Receives Request)

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class SecondServlet extends HttpServlet {

   public void doGet(HttpServletRequest req, HttpServletResponse res)
         throws IOException {

      res.setContentType("text/html");
      PrintWriter out = res.getWriter();

      out.println("<html><body>");
      out.println("<h2>Request forwarded to Second Servlet</h2>");
      out.println("</body></html>");
   }
}
```

HTML Form Code :

```
<!DOCTYPE html>
<html>
<head>
   <title>Servlet Forward Demo</title>
</head>
<body>

   <form action="first" method="GET">
      <input type="submit" value="Go to Second Servlet">
   </form>

</body>
</html>
```

web.xml

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
         version="3.1">
   <servlet>
      <servlet-name>FirstServlet</servlet-name>
      <servlet-class>FirstServlet</servlet-class>
   </servlet>
   <servlet-mapping>
      <servlet-name>FirstServlet</servlet-name>
      <url-pattern>/first</url-pattern>
   </servlet-mapping>
   <servlet>
      <servlet-name>SecondServlet</servlet-name>
      <servlet-class>SecondServlet</servlet-class>
   </servlet>
   <servlet-mapping>
      <servlet-name>SecondServlet</servlet-name>
      <url-pattern>/second</url-pattern>
   </servlet-mapping>
</web-app>
```

Practical - 10

// WRITE A PROGRAM FOR USER VALIDATION USING JSP.

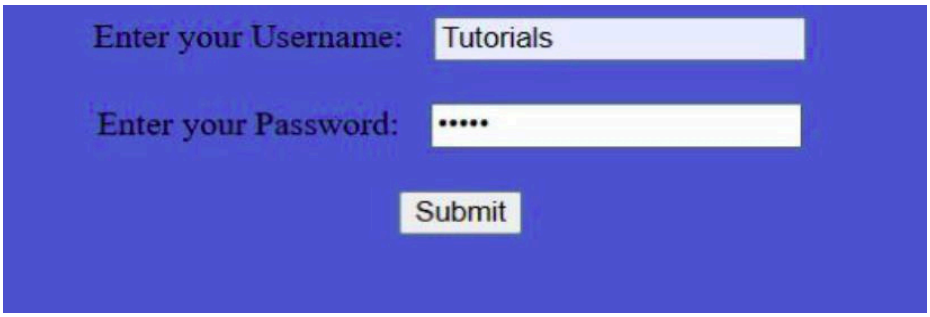
index.html (Login Page)

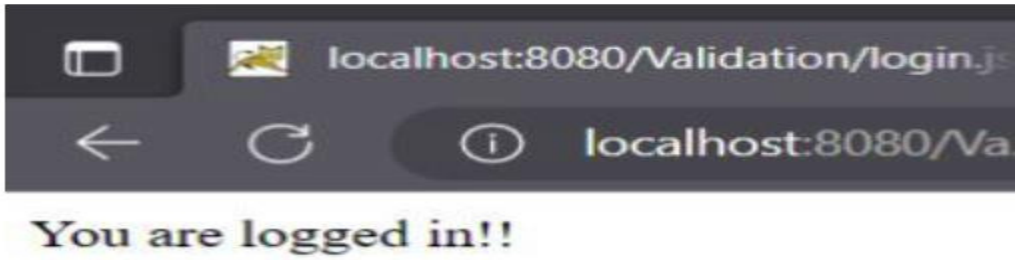
```
<!DOCTYPE html>
<html>
<head>
   <title>Login Page</title>
</head>
<body>
   <h2>User Login</h2>
   <form action="validate.jsp" method="post">
      Username: <input type="text" name="user"><br><br>
      Password: <input type="password" name="pass"><br><br>
      <input type="submit" value="Login">
   </form>
</body>
</html>
```

validate.jsp

```
<%@ page language="java" %>
<html>
<body>
<%
String user = request.getParameter("user");
String pass = request.getParameter("pass");
if(user != null && pass != null) {
   if(user.equals("admin") && pass.equals("1234")) {
%>
<h2>Login Successful</h2>
<h3>Welcome <%= user %></h3>
<%
   } else {
%>
<h2>Invalid Username or Password     </h2>
<%
   }
} else {
%>
<h2>Please enter credentials</h2>
<%
}
%>
</body>
</html>
```

OUTPUT:


Enter your Username:
Tutorials
Enter your Password:
Submit


localhost:8080/Validation/login.j
localhost:8080/Va
You are logged in!!

Practical - 11

// WRITE A PROGRAM FOR INSERTING DATA INTO TABLE USING JSP.

index.html (Login Page)

```
<!DOCTYPE html>
<html>
<head>
   <title>Insert Data</title>
</head>
<body>
   <h2>Insert Student Data</h2>
   <form action="insert.jsp" method="post">
      ID: <input type="text" name="id"><br><br>
      Name: <input type="text" name="name"><br><br>
      <input type="submit" value="Insert">
   </form>
</body>
</html>
```

insert.jsp

```
<%@ page import="java.sql.*" %>

<html>
<body>

<%
Connection con = null;
PreparedStatement ps = null;

String id = request.getParameter("id");
String name = request.getParameter("name");

try {

   Class.forName("com.mysql.cj.jdbc.Driver");

   con = DriverManager.getConnection(
      "jdbc:mysql://localhost:3306/test", "root", ""
   );

   String sql = "INSERT INTO student VALUES(?, ?)";
   ps = con.prepareStatement(sql);
   ps.setInt(1, Integer.parseInt(id));
   ps.setString(2, name);
```

```
 int i = ps.executeUpdate();

    if(i > 0) {
        out.println("<h2>Data Inserted Successfully     </h2>");
    } else {
        out.println("<h2>Insertion Failed     </h2>");
    }

} catch(Exception e) {
    out.println("<h2>Error: " + e.getMessage() + "</h2>");
} finally {
    if(ps != null) ps.close();
    if(con != null) con.close();
}
%>

</body>
</html>
```

OUTPUT :

```
+----+--------+
| id | name   |
+----+--------+
| 1  | Ram    |
| 2  | Shyam  |
+----+--------+
```

Practical - 12

// WRITE A JAVA PROGRAM TO DEMONSTRATE THE CONCEPT OF BEAN .

StudentBean.java

```
import java.io.Serializable;

public class StudentBean implements Serializable {

    private int id;
    private String name;

    public StudentBean() {}

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

BeanTest.java

```
public class BeanTest {

    public static void main(String[] args) {

        StudentBean student = new StudentBean();

        student.setId(101);
        student.setName("Ram");

        System.out.println("Student Details:");
        System.out.println("ID: " + student.getId());
        System.out.println("Name: " + student.getName());
    }
}
```

OUTPUT :

```
Student Details:
ID: 101
Name: Ram
```

Practical - 13

// WRITE A JAVA PROGRAM TO DEMONSTRATE THE CONCEPT OF RMI

RMI Program :

1. Create Remote Interface
2. Implement Interface (Server)
3. Create Server Program
4. Create Client Program

Hello.java (Remote Interface) :

```
import java.rmi.Remote;
import java.rmi.RemoteException;
public interface Hello extends Remote {
   String sayHello() throws RemoteException;
}
```

HelloImpl.java (Implementation) :

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;

public class HelloImpl extends UnicastRemoteObject implements Hello {

   public HelloImpl() throws RemoteException {
      super();
   }

   public String sayHello() throws RemoteException {
      return "Hello from RMI Server";
   }
}
```

RMIServer.java :

```
import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;
public class RMIServer {
   public static void main(String[] args) {
      try {
         HelloImpl obj = new HelloImpl();
         LocateRegistry.createRegistry(1099);
         Naming.rebind("rmi://localhost/HelloService", obj);
         System.out.println("RMI Server Started...");
      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

RMIClient.java :

```
import java.rmi.Naming;

public class RMIClient {

    public static void main(String[] args) {

        try {
            Hello stub = (Hello) Naming.lookup("rmi://localhost/HelloService");

            System.out.println("Response from Server: " + stub.sayHello());
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

OUTPUT :

```
RMI Server Started...
```

```
Response from Server: Hello from RMI Server
```

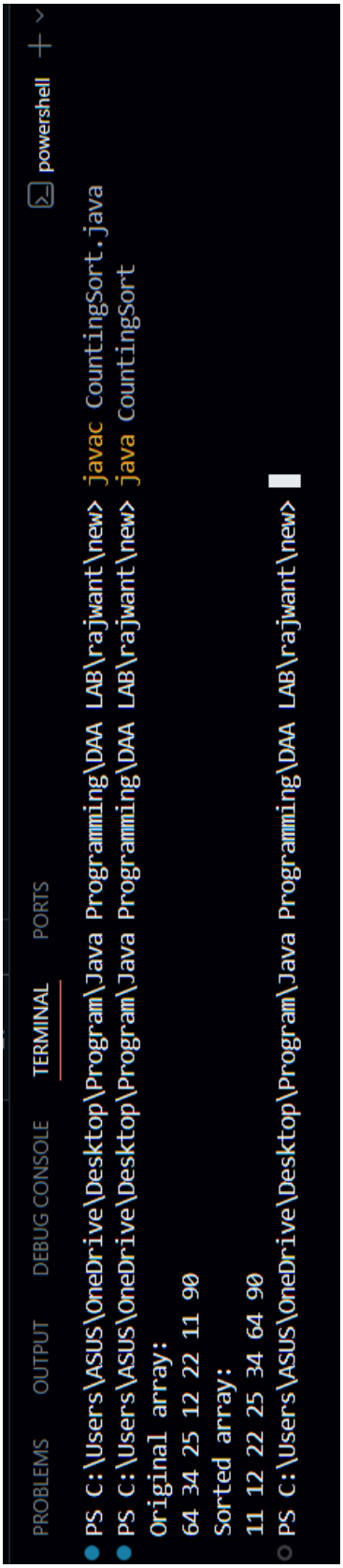
Practical - 7

// WRITE A PROGRAM TO IMPLEMENT COUNTING SORT .

CODE :

```
public class CountingSort {
    public static void countingSort(int[] arr, int n) {
        int max = arr[0];
        for (int i = 1; i < n; i++) {
            if (arr[i] > max)
                max = arr[i];
        }
        int[] count = new int[max + 1];
        int[] output = new int[n];

        for (int i = 0; i <= max; i++)
            count[i] = 0;

        for (int i = 0; i < n; i++)
            count[arr[i]]++;

        for (int i = 1; i <= max; i++)
            count[i] += count[i - 1];

        for (int i = n - 1; i >= 0; i--) {
            output[count[arr[i]] - 1] = arr[i];
            count[arr[i]]--;
        }

        for (int i = 0; i < n; i++)
            arr[i] = output[i];
    }

    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22, 11, 90};
        int n = arr.length;

        System.out.println("Original array:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }

        System.out.println();
        countingSort(arr, n);
        System.out.println("Sorted array:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

OUTPUT :



Practical - 9

// WRITE A PROGRAM TO IMPLEMENT FOR LONGEST COMMON SUBSEQUENCE .

CODE :

```
public class LCS {
    static int max(int a, int b) {
        return (a > b) ? a : b;
    }
    static int lcs(String X, String Y, int m, int n) {
        int[][] L = new int[m + 1][n + 1];

        for (int i = 0; i <= m; i++) {
            for (int j = 0; j <= n; j++) {

                if (i == 0 || j == 0)
                    L[i][j] = 0;
                else if (X.charAt(i - 1) == Y.charAt(j - 1))
                    L[i][j] = L[i - 1][j - 1] + 1;
                else
                    L[i][j] = max(L[i - 1][j], L[i][j - 1]);
            }
        }
        return L[m][n];
    }
    public static void main(String[] args) {

        String X = "AGGTAB";
        String Y = "GXTXAYB";

        int m = X.length();
        int n = Y.length();

        System.out.println("Length of LCS: " + lcs(X, Y, m, n));
    }
}
```

OUTPUT :

```
PS C:\Users\ASUS\OneDrive\Desktop\Program\Java Programming\DAA LAB\rajwant\new> javac LCS.java
PS C:\Users\ASUS\OneDrive\Desktop\Program\Java Programming\DAA LAB\rajwant\new> java LCS
Length of LCS: 4
PS C:\Users\ASUS\OneDrive\Desktop\Program\Java Programming\DAA LAB\rajwant\new>
```

Practical - 12

// WRITE A PROGRAM TO IMPLEMENT FOR HUFFMAN ALGORITHM.

CODE :

```
class Node {
    char data;
    int freq;
    Node left, right;

    Node(char data, int freq) {
        this.data = data;
        this.freq = freq;
        this.left = this.right = null;
    }
}

public class Huffman {

    static int findMin(Node[] arr, int n) {
        int min = 0;
        for (int i = 1; i < n; i++) {
            if (arr[i].freq < arr[min].freq)
                min = i;
        }
        return min;
    }

    static void printCodes(Node root, int[] code, int top) {
        if (root.left != null) {
            code[top] = 0;
            printCodes(root.left, code, top + 1);
        }

        if (root.right != null) {
            code[top] = 1;
            printCodes(root.right, code, top + 1);
        }

        if (root.left == null && root.right == null) {
            System.out.print(root.data + ": ");
            for (int i = 0; i < top; i++) {
                System.out.print(code[i]);
            }
            System.out.println();
        }
    }
```

```
static void buildHuffman(char[] data, int[] freq, int n) {
Node[] nodes = new Node[100];

for (int i = 0; i < n; i++) {
nodes[i] = new Node(data[i], freq[i]);
}

while (n > 1) {

int min1 = findMin(nodes, n);
Node left = nodes[min1];
nodes[min1] = nodes[n - 1];
n--;

int min2 = findMin(nodes, n);
Node right = nodes[min2];
nodes[min2] = nodes[n - 1];
n--;

Node newNode = new Node('$', left.freq + right.freq);
newNode.left = left;
newNode.right = right;

nodes[n] = newNode;
n++;
}

int[] code = new int[100];
printCodes(nodes[0], code, 0);
}
public static void main(String[] args) {

char[] data = {'a','b','c','d','e','f'};
int[] freq = {5,9,12,13,16,45};

int n = data.length;
buildHuffman(data, freq, n);
}
}
```

OUTPUT :

```
PS C:\Users\ASUS\OneDrive\Desktop\Program\Java Programming\DAA LAB\rajwant\new> javac Huffman.java
PS C:\Users\ASUS\OneDrive\Desktop\Program\Java Programming\DAA LAB\rajwant\new> java Huffman
f: 0
c: 100
d: 101
a: 1100
b: 1101
e: 111
```

Practical - 13

// WRITE A PROGRAM TO SIMULATE THE VARIOUS GRAPH TRAVERSING ALGORITHMS. (DIJKSTRA'S ALGORITHM).

CODE :

```
import java.util.Scanner;
public class Dijkstra {

 static final int MAX = 10;
 static final int INF = 9999;

 static int minDistance(int[] dist, boolean[] visited, int n) {
 int min = INF, index = -1;

 for (int i = 0; i < n; i++) {
 if (!visited[i] && dist[i] < min) {
 min = dist[i];
 index = i;
 }
 }
 return index;
 }
 static void dijkstra(int[][] graph, int start, int n) {

 int[] dist = new int[n];
 boolean[] visited = new boolean[n];

 for (int i = 0; i < n; i++) {
 dist[i] = INF;
 visited[i] = false;
 }

 dist[start] = 0;

 for (int i = 0; i < n - 1; i++) {

 int u = minDistance(dist, visited, n);
 if (u == -1) break;

 visited[u] = true;

 for (int v = 0; v < n; v++) {
 if (!visited[v] && graph[u][v] != 0 &&
 dist[u] + graph[u][v] < dist[v]) {

 dist[v] = dist[u] + graph[u][v];
 }
 }
 }
```

```
System.out.println("\nShortest distances:");
for (int i = 0; i < n; i++) {
System.out.println("Node " + i + " -> " + dist[i]);
}
}

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

int n, start;
int[][] graph = new int[MAX][MAX];

System.out.print("Enter number of nodes: ");
n = sc.nextInt();

System.out.println("Enter adjacency matrix:");
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
graph[i][j] = sc.nextInt();
}
}

System.out.print("Enter starting node: ");
start = sc.nextInt();

dijkstra(graph, start, n);
}
}
```

OUTPUT :

```
PS C:\Users\ASUS\OneDrive\Desktop\Program\Java Programming\DAA LAB\rajwant\new> javac Dijkstra.java
PS C:\Users\ASUS\OneDrive\Desktop\Program\Java Programming\DAA LAB\rajwant\new> java Dijkstra
Enter number of nodes: 5
Enter adjacency matrix:
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Enter starting node: 0

Shortest distances:
Node 0 -> 0
Node 1 -> 10
Node 2 -> 50
Node 3 -> 30
Node 4 -> 60
```

Practical - 14

// WRITE A PROGRAM TO SIMULATE THE VARIOUS TREE TRAVERSING ALGORITHMS. (BFS , DFS)

CODE :

```
import java.util.*;
class Node {
  int data;
  Node left, right;

  Node(int data) {
    this.data = data;
    left = right = null;
  }
}
public class TreeTraversal {

  static void preorder(Node root) {
    if (root == null) return;
    System.out.print(root.data + " ");
    preorder(root.left);
    preorder(root.right);
  }

  static void inorder(Node root) {
    if (root == null) return;
    inorder(root.left);
    System.out.print(root.data + " ");
    inorder(root.right);
  }

  static void postorder(Node root) {
    if (root == null) return;
    postorder(root.left);
    postorder(root.right);
    System.out.print(root.data + " ");
  }

  static void levelOrder(Node root) {
    Queue<Node> queue = new LinkedList<>();
    queue.add(root);

    while (!queue.isEmpty()) {
      Node temp = queue.poll();
      System.out.print(temp.data + " ");

      if (temp.left != null) queue.add(temp.left);
      if (temp.right != null) queue.add(temp.right);
    }
  }
```

```
  public static void main(String[] args) {

    Node root = new Node(1);
    root.left = new Node(2);
    root.right = new Node(3);
    root.left.left = new Node(4);
    root.left.right = new Node(5);
    root.right.right = new Node(6);

    Scanner sc = new Scanner(System.in);
    int choice;

    do {
      System.out.println("\n1.Preorder\n2.Inorder\n3.Postorder\n4.BFS\n5.Exit");
      System.out.print("Enter choice: ");
      choice = sc.nextInt();

      switch (choice) {
        case 1:
          System.out.print("Preorder: ");
          preorder(root);
          break;
        case 2:
          System.out.print("Inorder: ");
          inorder(root);
          break;
        case 3:
          System.out.print("Postorder: ");
          postorder(root);
          break;
        case 4:
          System.out.print("Level Order: ");
          levelOrder(root);
          break;
      }
      System.out.println();

    } while (choice != 5);
  }
}
```

OUTPUT :

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\ASUS\OneDrive\Desktop\Program\Java Programming\DAA LAB\rajwant\new> javac TreeTraversal.java
PS C:\Users\ASUS\OneDrive\Desktop\Program\Java Programming\DAA LAB\rajwant\new> java TreeTraversal

1.Preorder
2.Inorder
3.Postorder
4.BFS
5.Exit
Enter choice: 1
Preorder: 1 2 4 5 3 6

1.Preorder
2.Inorder
3.Postorder
4.BFS
5.Exit
Enter choice: 2
Inorder: 4 2 5 1 3 6

1.Preorder
2.Inorder
3.Postorder
4.BFS
5.Exit
Enter choice: 3
Postorder: 4 5 2 6 3 1

1.Preorder
2.Inorder
3.Postorder
4.BFS
5.Exit
Enter choice: 4
Level Order: 1 2 3 4 5 6

1.Preorder
2.Inorder
3.Postorder
4.BFS
5.Exit
Enter choice: 5

PS C:\Users\ASUS\OneDrive\Desktop\Program\Java Programming\DAA LAB\rajwant\new>
```